

# 《编译原理》实验报告

姓名：李经雄

学院：计算机科学与技术学院

专业：计算机科学与技术

邮箱：3220105346@zju.edu.cn

报告日期：2024. 3. 14

# Lab1 词法分析和语法分析

## 一 实验内容

### 1.1 词法分析器 Lexer

Lexer的实现比较简单，首先根据实验的简化SysY语法和Flex实现词法分析的功能。根据教程，利用yyval这样的内置变量传递信息，而我要做的仅仅只是描述好各种token的种类，各种关键字的token识别十分简单，只需要直接返回即可。通过正则表达式告诉Lexer我们合法的标识符和数字，注意0归类到8进制当中做特殊处理。当我们遇到数字时，通过strtol函数解析并把数值存入int\_val之中，再通过yyval传递给Parser。

### 1.2 语法分析器 Parser

Parser使用Bison自动化生成。基本规则部分只需要照抄简化SysY文法即可，值得注意的是如果遇到{ }这样的递归情形，需要设计一个新的非终结符来实现左递归，同时也需要给这个AST节点增加一个add\_dims()方法，以便在递归的时候调用将节点增添进去。

```
1 VarDef : IDENT { $$ = new VarDef($1); }
2       | IDENT "=" InitVal { $$ = new VarDef($1, shared_cast<InitVal>($3)); }
3       | IDENT ArrayDims { static_cast<VarDef*>($2)->set_ident($1); $$ = $2; }
4       | IDENT ArrayDims "=" InitVal { static_cast<VarDef*>($2)->set_ident($1);
5         static_cast<VarDef*>($2)->add_initVal(shared_cast<InitVal>($4)); $$ = $2; }
6       ;
7
7 ArrayDims : ArrayDim { $$ = new VarDef((shared_cast<IntConst>($1))->get_value()); }
8       | ArrayDim ArrayDims { static_cast<VarDef*>($2)->add_dim((shared_cast<IntConst>
9         ($1))->get_value()); $$ = $2; }
10
10 ArrayDim : "[" IntConst "]" { $$ = $2; }
11
12 InitVal : Exp { $$ = new InitVal(shared_cast<Node>($1)); }
13       | "{" "}" { $$ = new InitVal(); }
14       | "{" InitValList "}" { $$ = $2; }
15       ;
16
17 InitValList : InitVal { $$ = new InitVal(shared_cast<InitVal>($1)); }
18       | InitVal "," InitValList { static_cast<InitVal*>($3)->add_sub(shared_cast<InitVal>
19         ($1)); $$ = $3; }
```

比如实现VarDef的时候，就需要新建ArrayDims和InitValList这样的临时非终结符，才能便于表示左递归。

### 1.3 抽象语法树 AST

AST的实现应该算最难的部分，因为涉及到大量的指针操作，比Lexer和Parser部分复杂很多，这部分不仅在基础的代码上新增了几个节点，而且还要涉及数组的实现。下面是我新增的几个节点。

- Param：用于表示函数的形参，支持数组类型的形参
- ExpStmt：用于表示空语句

- PrimaryExp: 用于表示( Exp )括号的表达式
- IfStmt: 用于表示条件Stmt
- WhileStmt: 用于表示循环Stmt

以上节点用于补齐AST中的节点构造，更清晰地构建AST。数组实现以及初值实现的部分我放在第二节详细介绍。

## 二 实现亮点

### 2.1 语法树支持Array

需要支持数组的位置主要有VarDef LVal和Param三个部分，这里以VarDef为例：

```

1  class VarDef;
2  using VarDefPtr = std::shared_ptr<VarDef>;
3  class VarDef : public Node {
4  public:
5      std::string ident;
6      symbolPtr symbol;
7      // 支持数组和符合初值
8      std::vector<int> dims;
9      // 新增初值节点 如果没有 = 则为空
10     InitValPtr initVal;
11     // 构造函数1 仅有标识符(标量 无数组 无初值)
12     VarDef(const char *ident) : ident(ident), initVal(nullptr) {}
13     // 构造函数2 传入初始化器
14     VarDef(const char *ident, InitValPtr initVal) : ident(ident), initVal(initVal) {}
15     // 构造函数3 传入整维度信息
16     VarDef(const char *ident, int dim) : ident(ident), initVal(nullptr) {
17         dims.push_back(dim);
18     }
19     VarDef(int dim) : ident("temp"), initVal(nullptr) {
20         dims.push_back(dim);
21     }
22     void set_ident(const char *ident) { this->ident = ident; }
23     std::vector<int> get_dims() { return dims; }
24     void add_dim(int dim) { dims.insert(dims.begin(), dim); }
25     void add_initVal(InitValPtr initVal) { this->initVal = initVal; }
26     std::vector<NodePtr> get_children() override {
27         // 若想在打印或遍历AST时看 initVal 结构，把它返回
28         // dims 只是存 int 不是 NodePtr
29         if (initVal) {
30             return {initVal};
31         }
32         return {};
33     }
34 };

```

新创立一个dims的数组用于存储各个维度的大小，如果不是数组则没有元素，我们重载不同的构造函数方便在parser.y中进行调用，以适应不同的变量定义。注意这里添加维度的时候写法不是简单的push\_back而是从前面插入，这是因为设计的时候设计成了右递归的生成式，不过逻辑上是等价的。

复盘的时候发现这个实现也有很多不足的地方，因为没有独立的arrayDims类型的节点，所以只能将VarDef节点的构造下移到生成ArrayDims的规则中，这有些违背设计的初衷，更好的方法应该是将parser.y中的ArrayDims设计成一个数组类型的变量，这样在构建语法树的时候ArrayDims既可以不用独立成为一个节点使得AST过于臃肿，又可以很好的向上层传递维度的信息。

下面就不再列举形参和左值表达式的实现，思想比较类似。注意Param如果是数组的话第一个维度肯定是留空，所以我们使用一个特殊标记-1来标记。注意LVal因为[]方括号内可以是Exp表达式，在AST上难以直接计算表达式的值，我们只能将dims数组设计成为存储NodePtr的，这样可以返回指向维度信息的表达式指针，而在本节点只给出LVal到底有多少维。

## 2.2 语法树支持InitVal

初值的实现又是另外一个难点，因为InitVal作为一个节点却有两种形式：单个表达式或者符合花括号初值。这里选择了使用enum表示不同的模式，根据不同的生成式选择不同的构造函数，如果是表达式，则返回指向表达式的指针。如果是符合花括号初值，则将子初值存在一个数组中，返回的是所有的子表达式指针。这里我们将空花括号也看做符合初值，使用size=0进行标记。

```
1  class InitVal;
2  using InitValPtr = std::shared_ptr<InitVal>;
3  class InitVal : public Node {
4  public:
5      // 我们用一个 enum 来表示本节点哪种模式
6      enum class kind {
7          EXP,          // initVal 是一个表达式
8          BRACE         // initVal 是 { initVal, initVal, ... } 复合
9      } kind;
10     // 如果 kind==EXP, 我们存放 expr
11     NodePtr expr; // 这里可以是任意表达式节点
12     // 如果 kind==BRACE, 我们存放多个子InitVal
13     std::vector<InitValPtr> subInitVals;
14     // 构造1 表达式
15     InitVal(NodePtr e) : kind(kind::EXP), expr(e) {}
16     // 构造2 单个InitVal
17     InitVal(InitValPtr val) : kind(kind::BRACE), expr(nullptr) {
18         subInitVals.push_back(val);
19     }
20     // 构造3 花括号内为空
21     InitVal() : kind(kind::BRACE), expr(nullptr) {}
22
23     // 如果在解析时，需要反复添加 subInitVal，就加一个 add_sub(InitValPtr) 但是要添加到
subInitVals 头部
24     void add_sub(InitValPtr val) { subInitVals.insert(subInitVals.begin(), val); }
25     std::vector<NodePtr> get_children() override {
26         // 如果是表达式，就返回它
27         // 如果是复合，返回每个 subInitVal
28         if (kind == kind::EXP) {
29             if (expr) return {expr};
30             else return {};
31         } else {
32             std::vector<NodePtr> children;
33             for (auto &val : subInitVals) {
34                 children.push_back(val);
```

```
35     }
36     return children;
37 }
38 }
39 };
```

## 三 AI工具使用

---

- ChatGPT：将框架代码交给ChatGPT分析，并让他解释代码框架的思路和我所需要完成的任务。描述待完成工作并让ChatGPT生成可以采取的实现形式供参考
- Copilot：辅助完成代码实现和注释

## 四 建议

---

感觉可以将AST的实现放在完成Lexer和Parser之后，刚开始看到Parser的时候容易被AST的代码所影响，虽然两者确实关联十分紧密，但如果以从简单到困难的思路出发，分开实现可能更容易理解一些。